DATA ENGINEERING

OLAP vs OLTP Visual Guide

Transactions vs analytics.
Two workload shapes, two storage philosophies.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Two Workload Shapes

OLTP: many tiny writes. OLAP: few giant reads.

OLTP

- Thousands of small transactions/sec
- Read + write one row at a time
- Needs: ACID, low latency, concurrency

OLAP

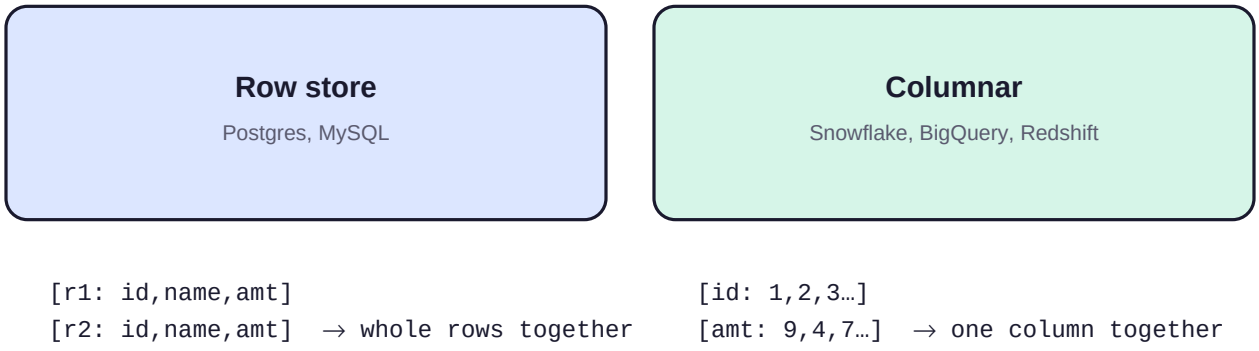
- Few queries, each scans millions of rows
- Read-mostly, bulk loads
- Needs: throughput, compression, aggregation

Example: the checkout page is OLTP (one order, milliseconds). The 'revenue by quarter' dashboard is OLAP (ten million orders, seconds).

STEP 2/6

Row vs Columnar Storage

Storage layout follows the access pattern.



OLTP reads whole rows (SELECT * WHERE id=…) → row storage wins.

OLAP reads few columns over many rows (SUM(amt)) → columnar

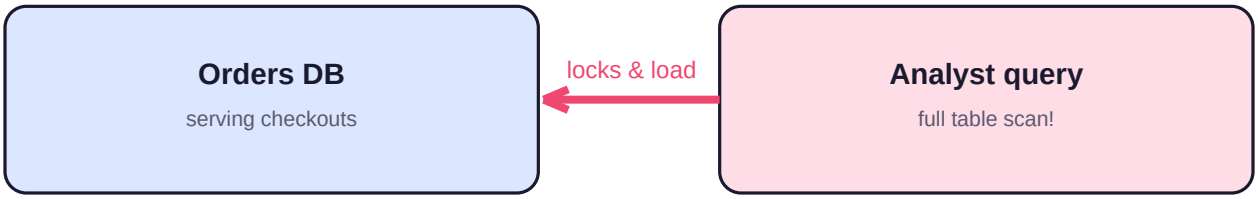
skips 90% of the data and compresses 10x better.

Same data, opposite layouts. This is why one database can't serve both workloads well.

STEP 3/6

Why You Can't Have One Database

Running analytics on the transactional DB breaks both.



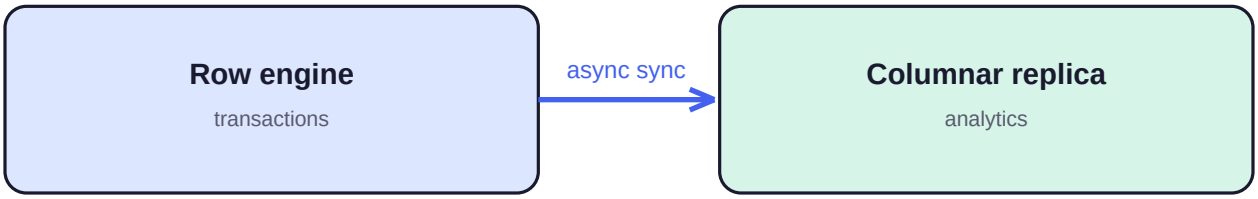
The classic outage: a BI query locks the orders table and checkout latency spikes. Separation is an operational requirement, not a preference.

- Analytical scans evict hot transactional data from memory
- Schema optimized for writes fights the schema optimized for reads
- Solution: replicate (CDC) into a dedicated analytical store

STEP 4/6

HTAP: Both in One System?

Hybrid transactional/analytical processing tries to serve both.



Systems: SingleStore, TiDB, AlloyDB, Snowflake Unistore.

Trick: one logical database, two physical engines with replication between them. Fresh analytics without ETL lag.

HTAP shines for operational analytics ('live fraud scores on fresh transactions'). For heavy historical analysis, a dedicated warehouse still wins.

STEP 5/6

Choosing Your Stores

Match the store to the workload, then connect them.

The standard pipeline: OLTP → CDC → warehouse. Fresh enough for dashboards (minutes), isolated enough to never threaten checkout latency.

Start boring: Postgres for transactions, a cloud warehouse for analytics, Debezium between them. Add HTAP only when ETL lag is the bottleneck.

STEP 6/6

The Comparison Matrix

One page to settle every 'which database' debate.

Dimension	OLTP	OLAP
Query pattern	Point lookups, small writes	Full scans, aggregations
Storage	Row-oriented	Columnar
Schema	Highly normalized (3NF)	Denormalized (star)
Concurrency	Thousands of users	Dozens of analysts
Example	Postgres, MySQL	Snowflake, BigQuery

HTAP exists for the overlap — but 'one database for everything' is still a trap for most teams. Separate, then integrate.

STEP ✓/✓

Key Takeaways

OLAP vs OLTP — Remember

- OLTP: many tiny transactions, row storage, normalized schema — built for checkout-speed writes.
- OLAP: few massive scans, columnar storage, star schemas — built for analytical throughput.
- Never run analytics on the transactional DB: scans lock tables and evict hot data.
- The standard bridge is CDC: OLTP → warehouse, minutes-fresh and fully isolated.
- HTAP (one system, two engines) fits operational analytics; heavy history still belongs in a warehouse.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev